

# Especificación Técnica: Proceso de Carga Masiva

Módulo de Horarios Escolares • Django + Pandas (SQLite / PostgreSQL)

## 1. Introducción y Arquitectura del Proceso

Para garantizar la agilidad demandada en la gestión de la institución, la app implementará un sistema de importación masiva de horarios mediante archivos **Excel (.xlsx)** o **CSV**. Dado que el colegio redefine su matriz horaria al inicio del ciclo lectivo (o ante reestructuraciones importantes), este proceso automatiza la creación de las relaciones en la base de datos, mitigando el error humano y reduciendo el tiempo de carga de días a minutos.

La arquitectura del proceso se divide en dos fases críticas bajo el principio de **Transaccionalidad Atómica**:

1. **Fase de Validación ("Dry-Run")**: El archivo es procesado íntegramente en memoria empleando la librería pandas. Se ejecutan validaciones sintácticas y lógicas sin persistir ningún dato. Si se encuentra un solo error, el proceso se aborta y se genera un reporte detallado.
2. **Fase de Inserción Eficiente ("Bulk Create")**: Una vez que el archivo está 100% libre de errores, se realiza una escritura masiva en la base de datos dentro de un bloque protegido, asegurando que el sistema nunca quede en un estado de carga parcial o corrupto.

## 2. Estructura de la Plantilla Excel / CSV

El personal administrativo utilizará una plantilla estandarizada. Para optimizar la experiencia de usuario y la fluidez, la app será flexible: si la **Materia** o el **Curso** no existen previamente en el sistema, la app los creará automáticamente en segundo plano. Sin embargo, el **Docente** y el **Bloque Horario** son mandatorios y preexistentes.

| Columna | Nombre Campo   | Tipo de Dato | Ejemplo      | Regla / Comportamiento                                               |
|---------|----------------|--------------|--------------|----------------------------------------------------------------------|
| A       | legajo_docente | TEXTO/NUM    | LEG-4412     | Obligatorio. Debe coincidir exactamente con el campo legajo.         |
| B       | día_semana     | ENTERO (1-5) | 2            | Días laborales: 1=Lunes, 2=Martes, 3=Miércoles, 4=Jueves, 5=Viernes. |
| C       | numero_bloque  | ENTERO       | 1            | Número identificador del Bloque Horario fijo de la institución.      |
| D       | materia        | TEXTO        | Matemática I | Obligatorio. Si no existe, la app la crea automáticamente.           |
| E       | curso          | TEXTO        | 4to Año A    | Obligatorio. Si no existe, la app lo crea automáticamente.           |

#### Estrategia de Visualización Limpia (UX)

Como los bloques se cargan de forma individual fila por fila (ej: Fila 1: Bloque 1 - Matemática; Fila 2: Bloque 2 - Matemática), el backend almacena la granularidad horaria exacta. Cuando el docente consulte su agenda, el Frontend agrupará visualmente estas filas consecutivas en un único módulo continuo de "Matemática (Bloque 1 y 2)" aplicando el algoritmo de unificación de celdas.

### 3. Matriz de Validaciones y Lógica de Negocio

El script de Python evaluará cada fila bajo tres niveles de control estricto. Cualquier fallo de nivel **Crítico** detendrá la operación.

- **Validación de Existencia del Docente (Crítico):** Si el legajo no existe en la tabla Docente, se rechaza el archivo indicando la fila exacta. No se autocrean docentes por seguridad de credenciales institucionales.
- **Validación de Bloque Horario (Crítico):** El número de bloque introducido debe existir en la configuración global de la institución.
- **Validación de Superposición / Duplicidad (Crítico):** El sistema verifica que el mismo docente no tenga dos materias asignadas en el mismo día y bloque horario ( $\text{Docente} \times \text{Día} \times \text{Bloque} = 1$ ).

#### **Política de Sobrescritura Automática**

Para dotar de agilidad al sistema, la carga masiva adoptará la política de **"Limpiar y Reemplazar"** o **"Upsert"**. Al subir un nuevo archivo consolidado, el sistema puede purgar el horario viejo del período lectivo actual e insertar el nuevo.

## 4. Implementación del Backend (Algoritmo Python/Django)

---

A continuación se detalla la lógica de negocio pura que se implementará en el controlador (View/API) de Django, utilizando pandas para eficiencia y `transaction.atomic()` para asegurar la integridad.

```

import pandas as pd
from django.db import transaction
from .models import Docente, BloqueHorario, Materia, Curso, HorarioDocente

def procesar_carga_masiva_horarios(archivo_excel):
    """
    Procesa la plantilla Excel de horarios, valida las reglas de negocio
    y realiza una inserción atómica masiva.
    """
    try:
        # 1. Leer el archivo Excel en un DataFrame
        df = pd.read_excel(archivo_excel)
        columnas = ['legajo_docente', 'dia_semana', 'numero_bloque', 'materia', 'curso']
        if not all(col in df.columns for col in columnas):
            return False, "Faltan columnas mandatorias en la plantilla."

        errores, nuevos_horarios = [], []
        dict_docentes = {d.legajo: d for d in Docente.objects.all()}
        dict_bloques = {b.numero: b for b in BloqueHorario.objects.all()}
        registros_vistos = set()

        # 2. FASE DE VALIDACIÓN
        for index, row in df.iterrows():
            fila = index + 2
            leg = str(row['legajo_docente']).strip()
            dia = int(row['dia_semana'])
            num_blq = int(row['numero_bloque'])
            mat = str(row['materia']).strip()
            cur = str(row['curso']).strip()

            if leg not in dict_docentes:
                errores.append(f"Fila {fila}: Docente '{leg}' no registrado.")
                continue

            if dia < 1 or dia > 5:
                errores.append(f"Fila {fila}: Día {dia} inválido (1-5).")
                continue

            if num_blq not in dict_bloques:
                errores.append(f"Fila {fila}: Bloque {num_blq} inexistente.")
                continue

            llave = (leg, dia, num_blq)
            if llave in registros_vistos:
                errores.append(f"Fila {fila}: Superposición bloque {num_blq}.")
                continue
            registros_vistos.add(llave)

            nuevos_horarios.append({
                'doc': dict_docentes[leg], 'dia': dia, 'blq': dict_bloques[num_blq],
                'mat': mat, 'cur': cur
            })

        if errores: return False, errores

        # 3. FASE DE INSERCIÓN ATÓMICA
        with transaction.atomic():
            objs = []

```

```

cache_mat = {m.nombre.lower(): m for m in Materia.objects.all()}
cache_cur = {c.nombre.lower(): c for c in Curso.objects.all()}

for item in nuevos_horarios:
    m_low, c_low = item['mat'].lower(), item['cur'].lower()
    if m_low not in cache_mat:
        cache_mat[m_low] = Materia.objects.create(nombre=item['mat'])
    if c_low not in cache_cur:
        cache_cur[c_low] = Curso.objects.create(nombre=item['cur'])

    objs.append(HorarioDocente(
        docente=item['doc'], dia=item['dia'], bloque=item['blq'],
        materia=cache_mat[m_low], curso=cache_cur[c_low]
    ))
HorarioDocente.objects.bulk_create(objs)

return True, f"Éxito: {len(objs)} registros procesados."
except Exception as e:
    return False, f"Error crítico: {str(e)}"

```

## 5. Flujo de Pantallas (UI/UX) para RRHH

Para el personal de Recursos Humanos, la interacción constará de un flujo compuesto por tres vistas:

1. **Vista de Carga:** Un panel limpio con una zona de Drag and Drop para el Excel y un botón para descargar la plantilla oficial.
2. **Vista de Feedback de Errores:** Si la validación falla, se muestra un listado estructurado de errores con fila y columna para rápida corrección.
3. **Vista de Éxito:** Pantalla de confirmación con métricas clave resumidas (docentes asignados, bloques cubiertos, nuevas materias creadas).